PROCESS SYNCHRONIZATION

A Comprehensive Comparison of Process Synchronization Mechanisms

Dekker • Peterson • Semaphore • Monitor • Readers–Writers

COURSE	Operating Systems / Embedded Systems
AUTHOR	Erfan Yazdani
INSTRUCTOR	Majid Shamshirgaran
UNIVERSITY	Eshragh University
DATE	12 November 2026
EDITION	English

Abstract and Contents

When several processes access shared data concurrently, a correct synchronization mechanism is required to avoid inconsistency. This article compares five well-known approaches: Dekker's and Peterson's software algorithms for two processes, semaphores as an operating-system primitive, monitors as a language-level construct, and the classical readers–writers problem. Each is presented with pseudocode, C code, a flowchart, an execution trace, worked examples and exam-style questions. Comparison tables cover mutual exclusion, progress, bounded waiting, starvation, deadlock, busy waiting, scalability and more, and charts visualise the differences.

Keywords: synchronization, critical section, mutual exclusion, Dekker, Peterson, semaphore, monitor, readers–writers

Contents

1 Introduction and the Critical-Section Problem
2 Dekker's Algorithm
3 Peterson's Algorithm
4 Semaphores
5 Monitors
6 Readers–Writers Problem
7 Comprehensive Comparison Tables
8 Charts
9 Examples and Exam Questions
10 Conclusion
11 References

1. Introduction and the Critical-Section Problem

In multiprocess and multithreaded systems, processes read and write shared data to exchange information or use common resources. If concurrent access is not controlled, the result depends on the exact interleaving of instructions, a phenomenon called a **race condition**.

A classic example: processes P1 and P2 execute `counter++` and `counter--` respectively on a shared variable `counter = 5`. At machine level each statement takes three steps: load into a register, modify the register, store back to memory. Depending on how the steps interleave, the final value may be 4, 5 or 6, although only 5 is correct.

Structure of a critical section

The part of the code that touches shared data is the **critical section (CS)**. Each process has four parts: entry, critical section, exit and remainder. The algorithms in this article are different ways of building the entry and exit sections.

Pseudocode — general structure

```
do {
    ENTRY SECTION        // ask for permission
    CRITICAL SECTION     // touches shared data
    EXIT SECTION         // give permission to others
    REMAINDER SECTION
} while (true);
```

The three correctness requirements

- **Mutual exclusion:** at most one process is inside the critical section at any time.
- **Progress:** if the critical section is free and some processes want to enter, only those processes take part in deciding who enters next, and the decision cannot be postponed indefinitely; a process in its remainder section must not block others.
- **Bounded waiting:** from the moment a process requests entry until it is granted, the number of times other processes enter first is bounded (no starvation).

Assumptions: nothing is assumed about the relative speed of processes, and reading or writing a simple variable is treated as an atomic action.

Table 1 — Classification of the methods

Method	Type	Implementation level	Requirements
Dekker	Software algorithm	User program	Shared memory only
Peterson	Software algorithm	User program	Shared memory only
Semaphore	Synchronization primitive	OS kernel / library	Atomic operations and a wait queue
Monitor	Structured construct	Language / library	Lock + condition variables
Readers–Writers	Classical problem	Design pattern	Solved with semaphores or monitors

Note. “Readers–writers” is a problem, not a primitive. In the comparison tables its column refers to the standard semaphore solution (reader priority); other solutions are noted separately.

In brief
A race condition arises when the result depends on the interleaving of operations on shared data. A correct solution must guarantee mutual exclusion, progress and bounded waiting without assuming anything about relative process speeds.

2. Dekker's Algorithm

Dekker's algorithm is the first correct software solution to the critical-section problem for two processes. It was devised by the Dutch mathematician T. J. Dekker and published by Dijkstra in 1965. It uses only shared memory and needs no special hardware instruction.

Idea

There are two shared variables: the array `flag[2]`, which tells whether each process wants to enter, and `turn`, which decides priority in case of conflict. A process first announces its interest; if the rival is also interested and the turn is not its own, it backs off (lowers its flag), waits until it is its turn, and tries again. This back-off is what removes the deadlock of simpler schemes.

Pseudocode — process Pi (j = 1 - i)

```
shared: boolean flag[2] = {false, false};
        int turn = 0;
Pi:
  repeat
    flag[i] = true;
    while (flag[j] == true) {
        if (turn != i) {          // not my turn: back off
            flag[i] = false;
            while (turn != i) ;   // wait for my turn
            flag[i] = true;
        }
    }
    // ---- CRITICAL SECTION ----
    turn = j;                     // hand priority to the other
    flag[i] = false;
    // ---- REMAINDER SECTION ----
  forever
```

C code (C11 atomics)

```
#include <stdatomic.h>
#include <stdbool.h>
atomic_bool flag[2] = { false, false };
atomic_int  turn = 0;

void dekker_enter(int i) {
    int j = 1 - i;
    atomic_store(&flag[i], true);
    while (atomic_load(&flag[j])) {
        if (atomic_load(&turn) != i) {
            atomic_store(&flag[i], false);
            while (atomic_load(&turn) != i) { /* busy wait */ }
            atomic_store(&flag[i], true);
        }
    }
}
void dekker_exit(int i) {
    atomic_store(&turn, 1 - i);
    atomic_store(&flag[i], false);
}
```

The code uses atomic variables with the default `seq_cst` ordering. On modern processors, replacing them with ordinary variables lets memory reordering break the algorithm.

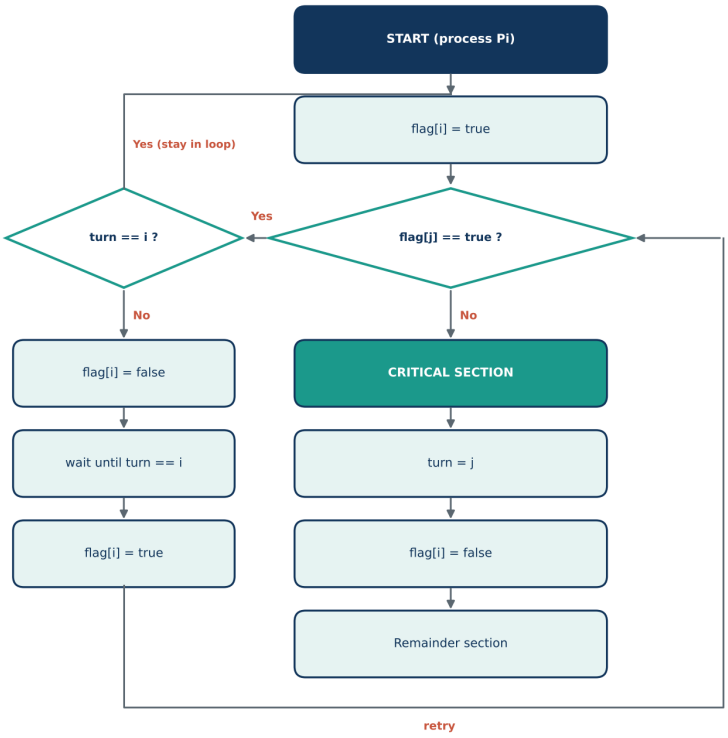


Figure 1 — Flowchart of Dekker's algorithm for process Pi

Execution trace

Assume turn = 0 and both processes want to enter at the same time:

Table 2 — Dekker trace (both processes request entry)

Step	P0	P1	f[0]	f[1]	turn
1	flag[0]=true	flag[1]=true	T	T	0
2	flag[1] is true; turn==0 → stays in loop	flag[0] is true; turn≠1 → flag[1]=false	T	F	0
3	flag[1] is false → enters CS	waits for turn==1	T	F	0
4	turn=1; flag[0]=false (exit)	turn==1 → flag[1]=true	F	T	1
5	remainder section	flag[0] is false → enters CS	F	T	1

Properties

- **Mutual exclusion:** holds; entry is possible only when flag[j] is seen false.
- **Progress:** holds; if the rival is not interested, flag[j]=false and the process enters regardless of turn.
- **Bounded waiting:** holds; on exit turn is given to the rival, so each process waits for at most one entry of the rival.
- **Drawbacks:** only two processes; busy waiting; subtle logic; needs memory ordering on modern hardware.

In brief
Dekker's algorithm (1965) uses flag[2] to announce intent and turn to resolve conflicts; the process without priority backs off by lowering its flag. It guarantees mutual exclusion, progress and bounded waiting, but only for two processes and with busy waiting.

3. Peterson's Algorithm

In 1981 G. L. Peterson published a much simpler two-process solution. The variables are the same (`flag[2]` and `turn`), but the logic is one of politeness: a process that wants to enter announces its interest and then gives the turn to the rival. If both want to enter at once, the one that wrote turn last has yielded and waits.

Pseudocode — process Pi (j = 1 - i)

```
shared: boolean flag[2] = {false, false};
        int turn;
Pi:
  repeat
    flag[i] = true;                       // I am interested
    turn = j;                             // but you go first
    while (flag[j] == true && turn == j) ; // wait
    // ---- CRITICAL SECTION ----
    flag[i] = false;                      // I am done
    // ---- REMAINDER SECTION ----
  forever
```

C code (C11 atomics)

```
#include <stdatomic.h>
#include <stdbool.h>
atomic_bool flag[2] = { false, false };
atomic_int  turn;

void peterson_enter(int i) {
    int j = 1 - i;
    atomic_store(&flag[i], true);
    atomic_store(&turn, j);
    while (atomic_load(&flag[j]) && atomic_load(&turn) == j) {
        /* busy wait */
    }
}
void peterson_exit(int i) {
    atomic_store(&flag[i], false);
}
```

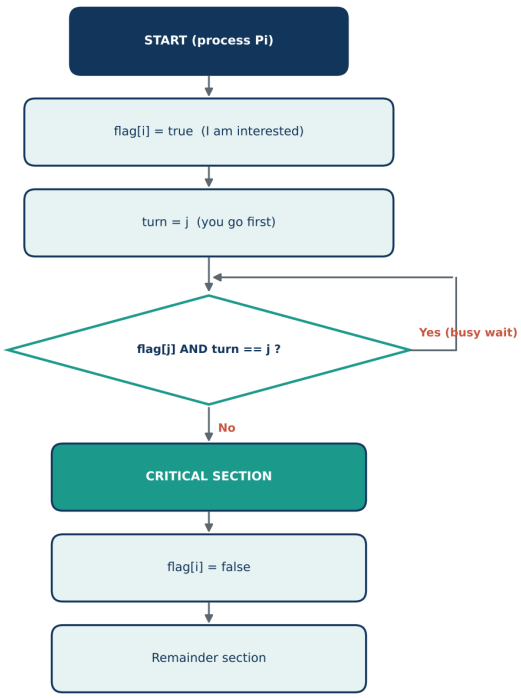


Figure 2 — Flowchart of Peterson's algorithm

Short proof of mutual exclusion

Suppose P0 and P1 are both in the critical section. Each saw the loop condition false before entering. P0 saw either `flag[1]` false or `turn==0`; but P1 had already written `flag[1]=true` and keeps it until exit, so P0 must have seen `turn==0`. Symmetrically P1 must have seen `turn==1`. Since turn is a single variable and the last writer decides, both cannot hold at once — a contradiction.

Execution trace

Table 3 — Peterson trace (both processes request entry)

Step	P0	P1	f[0]	f[1]	turn
1	flag[0]=true	—	T	F	?
2	—	flag[1]=true	T	T	?
3	turn=1	—	T	T	1
4	—	turn=0 (last writer)	T	T	0
5	flag[1]&&turn==1? false → enters	flag[0]&&turn==0? true → waits	T	T	0
6	flag[0]=false (exit)	condition false → enters	F	T	0

Properties and notes

- **Mutual exclusion, progress and bounded waiting** all hold; the rival enters at most once before you.
- **Drawbacks:** only two processes, busy waiting, and it needs sequentially consistent memory; without seq_cst or a memory fence, the store buffer can reorder flag[i]=true and the read of flag[j], letting both processes enter.
- Generalisation to N processes is possible with the filter lock or the Bakery algorithm, at higher cost and complexity.

In brief
Peterson's algorithm (1981) simplifies Dekker's idea: a process sets its flag and then yields the turn to the other. The last writer of turn waits. It satisfies all three requirements for two processes in a few lines, but needs sequentially consistent memory and busy waiting.

4. Semaphores

Semaphores were introduced by Dijkstra in 1965. A semaphore is an integer S modified only through two atomic operations: **wait** (Dijkstra's P, *proberen*) and **signal** (V, *verhogen*). Unlike Dekker and Peterson, semaphores work for N processes, and in the blocking implementation a waiting process is put to sleep in a queue instead of spinning, which frees the processor.

Definition and blocking implementation

Pseudocode — semaphore operations

```
typedef struct { int value; queue list; } semaphore;

wait(S):     // P, down  (executed atomically)
    S.value = S.value - 1;
    if (S.value < 0) {
        add this process to S.list;
        block();                 // sleep: no busy waiting
    }

signal(S):   // V, up  (executed atomically)
    S.value = S.value + 1;
    if (S.value <= 0) {
        remove a process P from S.list;
        wakeup(P);
    }
```

In this implementation a negative value of S equals the number of waiting processes. Two common kinds exist: the **binary semaphore** (mutex) with value 0 or 1 for mutual exclusion, and the **counting semaphore** for multi-instance resources.

Pseudocode — mutual exclusion

```
semaphore mutex = 1;
Pi:
  repeat
    wait(mutex);
    // ---- CRITICAL SECTION ----
    signal(mutex);
    // ---- REMAINDER SECTION ----
  forever
```

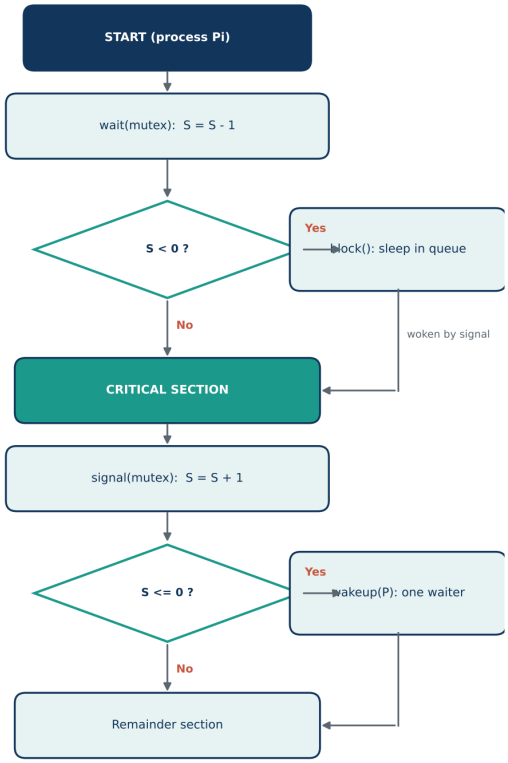


Figure 3 — Flowchart of semaphore usage (wait / signal)

Example: bounded-buffer producer–consumer

Three semaphores are needed: `empty` (free slots, initially N), `full` (filled slots, initially 0) and `mutex` (exclusive access to the buffer).

Pseudocode

```
semaphore mutex = 1, empty = N, full = 0;

Producer:                    Consumer:
  repeat                       repeat
    item = produce();            wait(full);
    wait(empty);                 wait(mutex);
    wait(mutex);                 item = remove();
    insert(item);                signal(mutex);
    signal(mutex);               signal(empty);
    signal(full);                consume(item);
  forever                      forever
```

C code (POSIX threads + semaphores)

```
#include <pthread.h>
#include <semaphore.h>
#define N 8

int buf[N], in = 0, out = 0;
sem_t empty, full, mutex;

void *producer(void *arg) {
    for (int i = 0; i < 100; i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
        buf[in] = i; in = (in + 1) % N;
        sem_post(&mutex);
        sem_post(&full);
    }
    return NULL;
}
void *consumer(void *arg) {
```

```
    for (int i = 0; i < 100; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        int x = buf[out]; out = (out + 1) % N;
        sem_post(&mutex);
        sem_post(&empty);
        (void)x;  /* use the item here */
    }
    return NULL;
}
int main(void) {
    pthread_t p, c;
    sem_init(&empty, 0, N);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    return 0;
}
```

Properties and pitfalls

- Mutual exclusion and progress hold with correct use. **Bounded waiting** depends on the queue policy: a strong semaphore (FIFO queue) guarantees it, a weak one (arbitrary wake-up) does not.
- **Deadlock:** a wrong order of operations, e.g. wait(mutex) before wait(empty) in the producer, can deadlock.
- **Error-prone:** forgetting a signal or swapping wait and signal breaks mutual exclusion or blocks processes forever; it is an unstructured, low-level tool.
- **Priority inversion** may occur and calls for protocols such as priority inheritance.

In brief
A semaphore is an integer manipulated only by atomic wait (P) and signal (V). Blocking semaphores avoid busy waiting and work for any number of processes, but are low-level and error-prone. Bounded waiting holds only with FIFO (strong) semaphores.

5. Monitors

Monitors were proposed by C. A. R. Hoare (1974) and Brinch Hansen to reduce the error-proneness of semaphores. A monitor is an abstract data type that holds shared data and the procedures that access it, and the language or compiler guarantees that at most one process is active inside its procedures at any time, so the programmer does not write wait/signal for mutual exclusion.

For conditional synchronization (e.g. “the buffer is not empty”) a monitor uses **condition variables** with two operations: `x.wait()` blocks the process and releases the monitor lock, and `x.signal()` wakes one waiting process. Unlike a semaphore signal, a condition signal with no waiter has no effect and is not remembered.

Two signal semantics

- **Hoare (signal-and-wait):** the signalling process hands the monitor to the woken one immediately; the condition is guaranteed to hold, so *if* suffices.
- **Mesa (signal-and-continue):** the signaller continues and the woken process re-acquires the lock later; the condition may have changed, so a *while* loop is required. Java, pthreads and most real systems follow Mesa.

Pseudocode — bounded-buffer monitor

```
monitor BoundedBuffer {
    int buf[N]; int count = 0, in = 0, out = 0;
    condition notFull, notEmpty;

    procedure put(x) {
        while (count == N) notFull.wait();
        buf[in] = x; in = (in + 1) % N; count++;
        notEmpty.signal();
    }
    procedure get() {
        while (count == 0) notEmpty.wait();
        x = buf[out]; out = (out + 1) % N; count--;
        notFull.signal();
        return x;
    }
}  // at most one process is active inside at any time
```

C code (pthread mutex + condition variables)

```
#include <pthread.h>
#define N 8
static int buf[N], in = 0, out = 0, count = 0;
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;   /* monitor lock */
static pthread_cond_t notFull  = PTHREAD_COND_INITIALIZER;
static pthread_cond_t notEmpty = PTHREAD_COND_INITIALIZER;

void put(int x) {
    pthread_mutex_lock(&m);                              /* enter monitor */
    while (count == N) pthread_cond_wait(&notFull, &m);
    buf[in] = x; in = (in + 1) % N; count++;
    pthread_cond_signal(&notEmpty);
    pthread_mutex_unlock(&m);                            /* exit monitor */
}
int get(void) {
    pthread_mutex_lock(&m);
    while (count == 0) pthread_cond_wait(&notEmpty, &m);
    int x = buf[out]; out = (out + 1) % N; count--;
    pthread_cond_signal(&notFull);
    pthread_mutex_unlock(&m);
    return x;
}
```

In Java the same pattern is written with `synchronized` and `wait()/notifyAll()`. C has no built-in monitor; here it is **emulated** with a lock and condition variables.

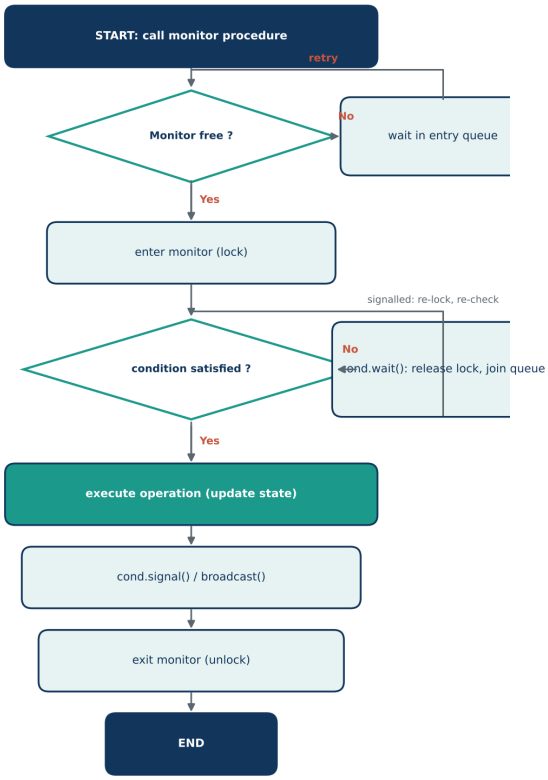


Figure 4 — Flowchart of monitor entry/exit with a condition variable

Properties and pitfalls

- **Automatic, structured mutual exclusion;** the safest and most readable tool in this article.
- **Deadlock** is still possible, e.g. through nested monitor calls that hold another lock.
- **Starvation** is possible under Mesa semantics if woken processes repeatedly find the condition false and the queue is not fair.
- Needs language or library support; has lock and context-switch costs.

In brief
A monitor bundles shared data and its procedures; the language guarantees that only one process is active inside at a time. Condition variables provide conditional synchronization; a signal with no waiter is lost. With Mesa semantics, always re-check the condition in a while loop.

6. Readers–Writers Problem

The readers–writers problem was posed by Courtois, Heymans and Parnas in 1971. A shared data object (e.g. a database) is used by **readers**, which only read and may run concurrently, and **writers**, which need fully exclusive access: no other writer and no reader may be active together with a writer. Unlike plain mutual exclusion, concurrency among readers is allowed and raises throughput.

Three variants

Table 4 — Variants of the readers–writers problem

Variant	Rule	Consequence
First — reader priority	While some reader is reading, a new reader may enter	Possible writer starvation
Second — writer priority	When a writer is waiting, no new reader may enter	Possible reader starvation
Third — fair	Entry in arrival order (FIFO) for all requests	No starvation; less concurrency

Semaphore solution (reader priority)

`rw_mutex` controls access to the data, `read_count` holds the number of active readers, and `mutex` protects read_count. The first reader locks writers out and the last reader lets them in again.

Pseudocode

```
semaphore rw_mutex = 1, mutex = 1;
int read_count = 0;

Writer:                          Reader:
  wait(rw_mutex);                  wait(mutex);
  // WRITE                         read_count++;
  signal(rw_mutex);                if (read_count == 1) wait(rw_mutex);
                                   signal(mutex);
                                   // READ
                                   wait(mutex);
                                   read_count--;
                                   if (read_count == 0) signal(rw_mutex);
                                   signal(mutex);
```

C code (POSIX semaphores)

```
#include <semaphore.h>
sem_t rw_mutex, mutex;   /* both initialised to 1 */
int read_count = 0;

void writer(void) {
    sem_wait(&rw_mutex);
    /* ... write shared data ... */
    sem_post(&rw_mutex);
}
void reader(void) {
    sem_wait(&mutex);
    if (++read_count == 1) sem_wait(&rw_mutex);   /* first reader */
    sem_post(&mutex);

    /* ... read shared data ... */

    sem_wait(&mutex);
    if (--read_count == 0) sem_post(&rw_mutex);   /* last reader */
    sem_post(&mutex);
}
```

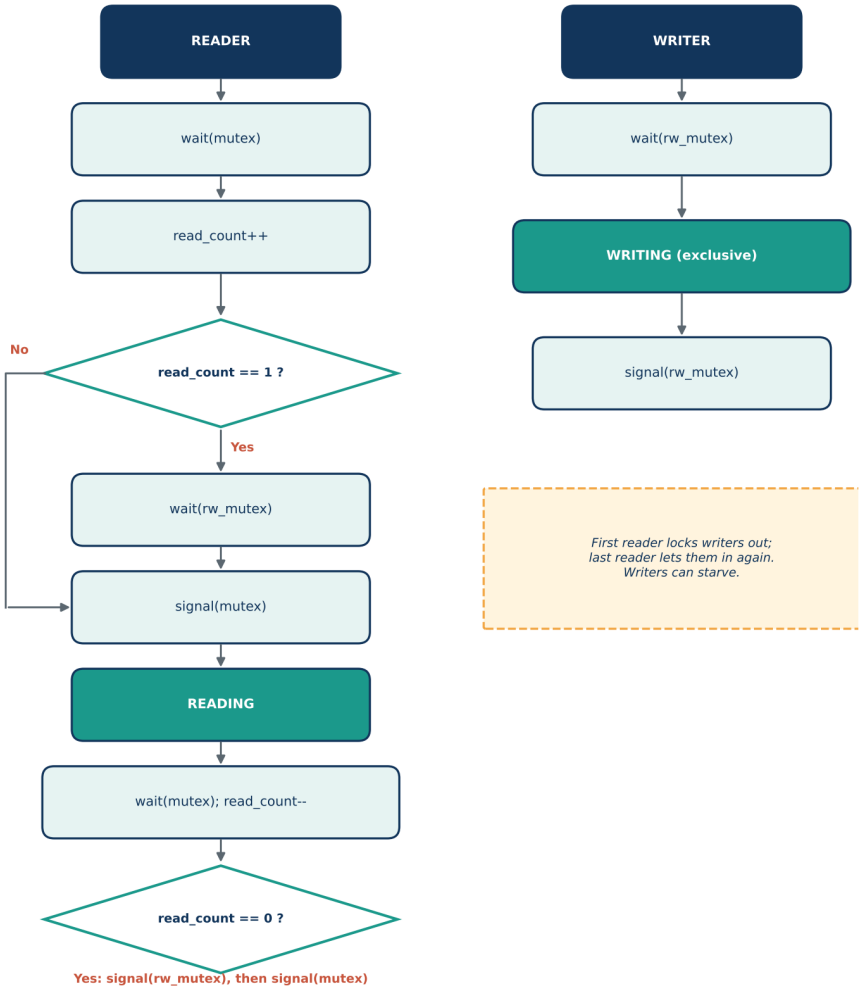


Figure 5 — Flowchart of reader and writer (semaphore solution)

Fair version with semaphores

Adding a queue semaphore `service` that every request passes through preserves arrival order:

Pseudocode — third problem

```
semaphore service = 1, rw_mutex = 1, mutex = 1;  int read_count = 0;

Reader:                          Writer:
  wait(service);                   wait(service);
  wait(mutex);                     wait(rw_mutex);
  if (++read_count == 1)           signal(service);
      wait(rw_mutex);              // WRITE
  signal(service);                 signal(rw_mutex);
  signal(mutex);
  // READ
  wait(mutex);
  if (--read_count == 0) signal(rw_mutex);
  signal(mutex);
```

Monitor solution (writer priority)

Pseudocode

```
monitor ReadersWriters {
    int readers = 0, waitingWriters = 0; boolean writing = false;
    condition okToRead, okToWrite;

    procedure startRead() {
        while (writing || waitingWriters > 0) okToRead.wait();
        readers++;
    }
```

```
    procedure endRead() {
        readers--;
        if (readers == 0) okToWrite.signal();
    }
    procedure startWrite() {
        waitingWriters++;
        while (writing || readers > 0) okToWrite.wait();
        waitingWriters--; writing = true;
    }
    procedure endWrite() {
        writing = false;
        if (waitingWriters > 0) okToWrite.signal();
        else okToRead.broadcast();
    }
}
```

C code (pthread monitor)

```
#include <pthread.h>
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t okToRead  = PTHREAD_COND_INITIALIZER;
static pthread_cond_t okToWrite = PTHREAD_COND_INITIALIZER;
static int readers = 0, writing = 0, waitingWriters = 0;

void start_read(void) {
    pthread_mutex_lock(&m);
    while (writing || waitingWriters > 0) pthread_cond_wait(&okToRead, &m);
    readers++;
    pthread_mutex_unlock(&m);
}
void end_read(void) {
    pthread_mutex_lock(&m);
    if (--readers == 0) pthread_cond_signal(&okToWrite);
    pthread_mutex_unlock(&m);
}
void start_write(void) {
    pthread_mutex_lock(&m);
    waitingWriters++;
    while (writing || readers > 0) pthread_cond_wait(&okToWrite, &m);
    waitingWriters--; writing = 1;
    pthread_mutex_unlock(&m);
}
void end_write(void) {
    pthread_mutex_lock(&m);
    writing = 0;
    if (waitingWriters > 0) pthread_cond_signal(&okToWrite);
    else pthread_cond_broadcast(&okToRead);
    pthread_mutex_unlock(&m);
}
```

POSIX also provides a ready-made reader–writer lock (`pthread_rwlock_t`), which is preferable to handwritten code in practice.

> **In brief**
> Readers may share the data; a writer needs exclusive access. The first solution (reader priority) can starve writers, the second (writer priority) can starve readers, and the third uses a service queue to be fair. Both semaphores and monitors can implement all three.

7. Comprehensive Comparison Tables

This section compares the five methods on correctness parameters (Table 5) and engineering parameters (Table 6). The “Readers–Writers” column refers to the standard semaphore solution (reader priority) unless stated otherwise.

Table 5 — Correctness and behaviour

Parameter	Dekker	Peterson	Semaphore	Monitor	Readers-Writers
Mutual exclusion	✔	✔	✔ (if used correctly)	✔ (automatic)	✔ (writer vs. all)
Progress	✔	✔	✔	✔	✔
Bounded waiting	✔ (at most once)	✔ (at most once)	Only with FIFO queue	Usually with FIFO queue	Variant 1 ✘, variant 3 ✔
Starvation	None	None	Possible (weak semaphore)	Possible (Mesa)	Writer (var. 1) / reader (var. 2)
Deadlock	None	None	Possible with wrong order	Possible (nested calls)	Possible with wrong order
Busy waiting	Yes	Yes	No (blocking type)	No	No
Fairness	Nearly fair (2 procs)	Fair (2 procs)	Depends on queue	Depends on cond. queue	Variant 3 is fair
Reader concurrency	—	—	—	—	Yes (goal of the problem)

Table 6 — Engineering and practical aspects

Parameter	Dekker	Peterson	Semaphore	Monitor	Readers–Writers
Year / inventor	1965, Dekker / Dijkstra	1981, Peterson	1965, Dijkstra	1974, Hoare / Brinch Hansen	1971, Courtois et al.
Type	Software algorithm	Software algorithm	OS primitive	Language construct	Classical problem
Number of processes	2	2 (generalised: Filter)	N	N	N readers + N writers
Hardware / OS need	Memory only (+ memory ordering)	Memory only (+ memory ordering)	Atomic ops + OS queue	Lock + condition variable	Semaphore or monitor
Shared variables	3	3	1 (mutex)	0 explicit (internal lock)	3 (rw_mutex, mutex, read_count)
Abstraction level	Low	Low	Medium	High	Design pattern
Ease of use	Hard	Fairly simple	Medium	Easy	Medium
Error-proneness	High	Medium	High	Low	Medium
Run-time cost	High CPU use	High CPU use	Context switch when waiting	Lock + context switch	Same as base tool
Typical use	Teaching, theory	Teaching, theory, simple systems	Kernels, threads, counted resources	Java, C#, libraries	Databases, caches

Summary of the tables

Dekker and Peterson suit teaching and the theory of correctness; semaphores suit more than two processes without busy waiting; monitors give safer and more readable code; and the readers–writers pattern fits data with many readers and few writers, with the policy (reader, writer or fair) chosen to match the workload.

8. Charts

The charts below visualise several aspects of the comparison. The scores in Figures 6 and 7 are the **author's qualitative assessment** based on structural properties of each method (1 = weak, 5 = strong), not the result of empirical measurements.

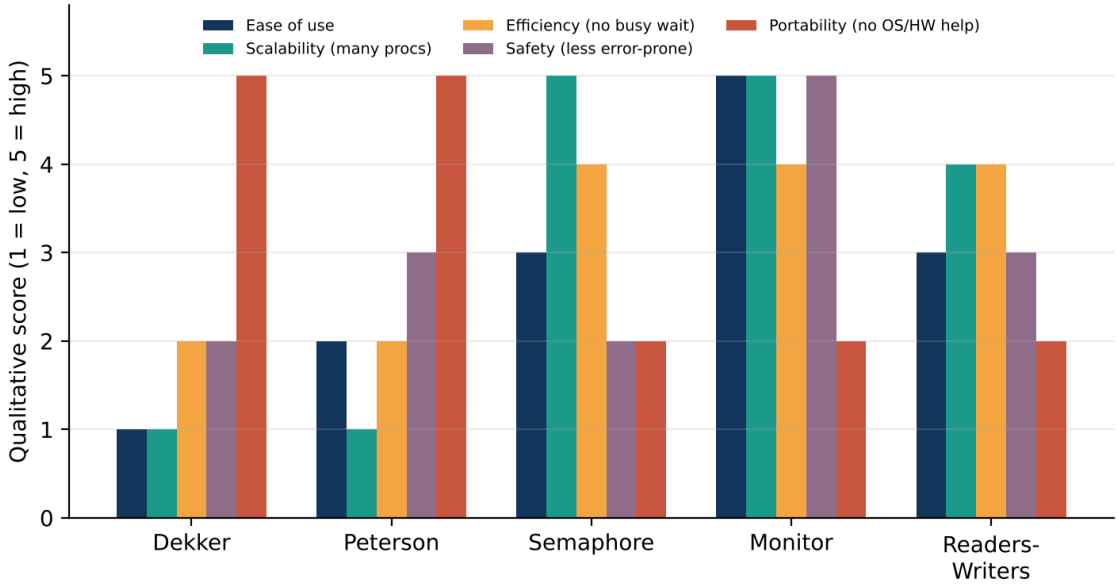


Figure 6 — Qualitative comparison on five criteria (bar chart)

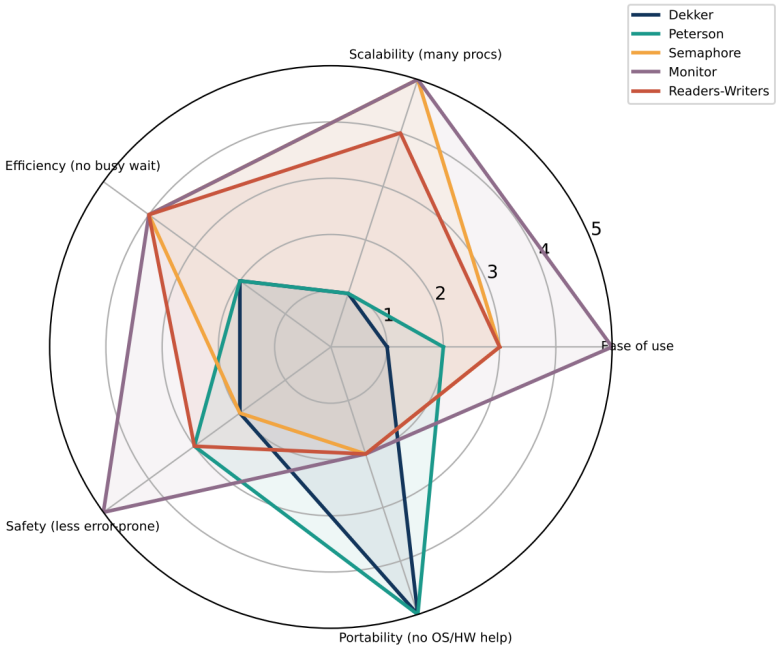


Figure 7 — Radar chart of the same criteria

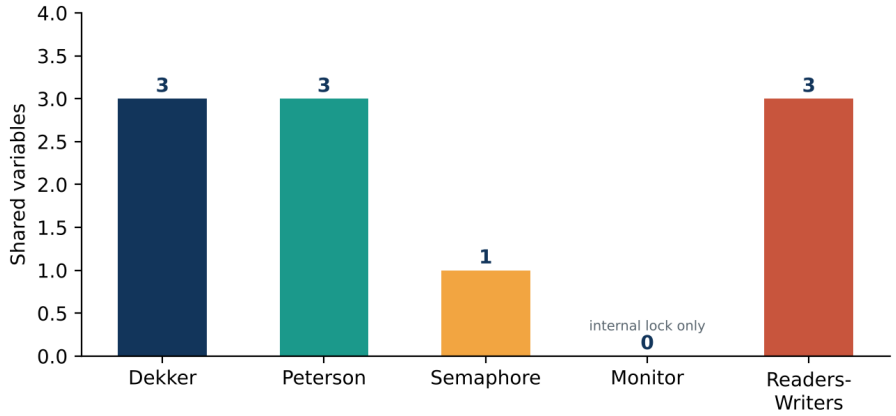


Figure 8 — Shared variables in the presented implementations

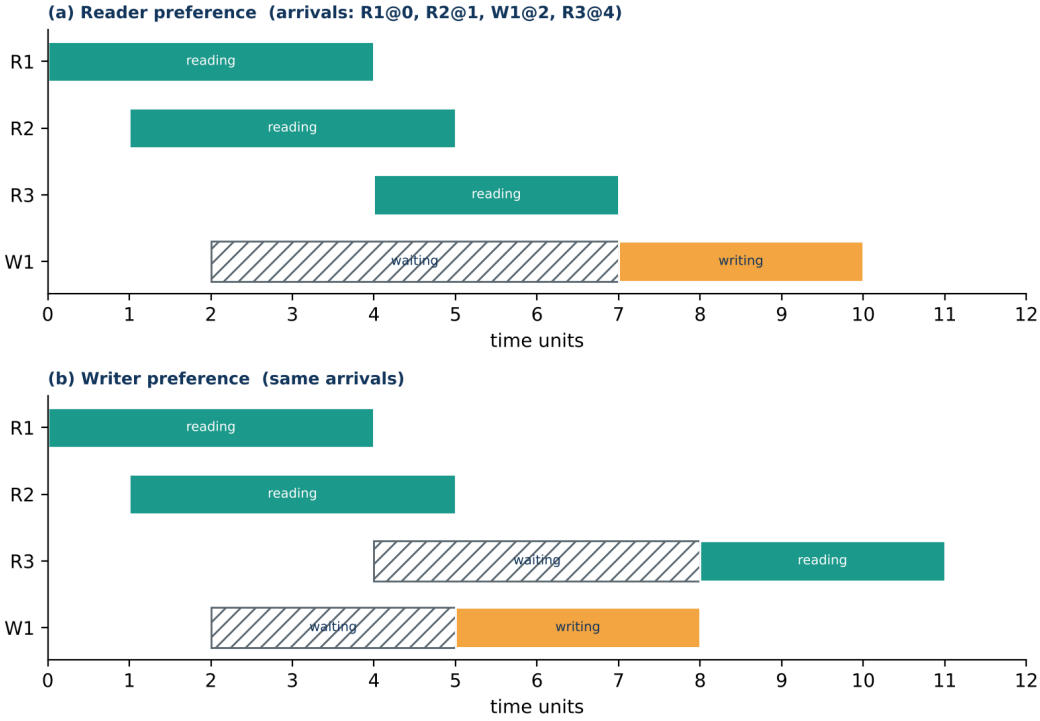


Figure 9 — Sample timeline: reader preference vs. writer preference

In Figure 9 (an illustrative scenario, not a measurement) R1 arrives at time 0, R2 at 1, W1 at 2 and R3 at 4. With reader priority R3 is admitted because readers are already reading, and W1 waits until time 7. With writer priority R3 is held behind W1, and W1 writes once R1 and R2 finish (time 5).

9. Examples and Exam Questions

For each of the five methods, one worked example and several questions with answers are given.

9.1 Dekker

Worked example. If turn = 1 and only P0 wants to enter, what happens? P0 writes flag[0]=true, sees flag[1] false and enters immediately, even though the turn belongs to P1. This is the progress property: turn matters only when there is a conflict.

Question 1: What happens if the statement flag[i] = false inside the loop is removed from Dekker's algorithm?

Answer: Deadlock. Suppose turn = 0 and both flags are true. P1, since turn ≠ 1, waits for turn == 1 but keeps its flag true; P0 sees flag[1] true and spins in the while loop. turn changes only after P0 exits, so neither makes progress.

Question 2: For how many processes does Dekker's algorithm work, and does it have bounded waiting?

Answer: Only 2 processes. Yes, it has bounded waiting: after exit, turn is given to the rival, so each process waits for at most one entry of the rival.

9.2 Peterson

Worked example. If P1 is in its remainder section, flag[1]=false. P0 writes its flag and turn=1, but the condition flag[1] && turn==1 is false, so it enters: a lone requester is never blocked.

Question 3: If the two entry statements are swapped (first turn = j, then flag[i] = true), is mutual exclusion violated? Build a scenario.

Answer: Yes. P0: turn=1; P1: turn=0; P1: flag[1]=true; P1 sees flag[0] false and enters. Then P0: flag[0]=true; the condition flag[1] && turn==1 is false because turn==0, so P0 enters too, and both are inside the critical section.

Question 4: Why can Peterson's algorithm fail on modern processors without a memory fence?

Answer: The processor or compiler may reorder the write of flag[i]=true and the read of flag[j] (especially with a store buffer). Both processes may then read the other's flag as false and enter. The fix is seq_cst atomics or a memory fence.

9.3 Semaphore

Worked example — counting semaphore with initial value 2. Four processes A..D execute wait(S) in order, then A and B execute signal(S): A: S=1; B: S=0; C: S=-1 (blocked); D: S=-2 (blocked); A: signal → S=-1, C wakes; B: signal → S=0, D wakes. A negative value shows the number of waiters.

Question 5: Using a semaphore, guarantee that statement S1 of P1 always runs before S2 of P2.

Answer: Create semaphore sync with value 0. P1: S1; signal(sync); and P2: wait(sync); S2;. If P2 arrives first it blocks until P1 signals.

Question 6: Show how a wrong order of wait calls creates a deadlock.

Answer: With two semaphores A=1 and B=1: P1: wait(A); wait(B); and P2: wait(B); wait(A);. If P1 takes A and P2 takes B, each waits for the other forever. Fix: always acquire semaphores in a fixed global order.

9.4 Monitor

Worked example — buffer of capacity 2. put(x) and put(y) fill the buffer (count=2). put(z) blocks in notFull.wait() and releases the lock. get() removes an item, count=1, and notFull.signal() wakes the waiting producer; it re-checks count==N, finds it false and inserts z.

Question 7: Why must a Mesa-style monitor wait on a condition with while rather than if?

Answer: Between the signal and the re-acquisition of the lock, another process may enter and change the condition (or a spurious wake-up may occur). while re-checks the condition after waking.

Question 8: What is the difference between a semaphore signal and a condition-variable signal?

Answer: A semaphore signal increments the counter and is remembered; a condition signal with no waiter has no effect and is lost.

9.5 Readers–Writers

Worked example — reader priority. R1 enters (read_count=1, takes rw_mutex). W1 blocks on wait(rw_mutex). R2 arrives: read_count=2, not 1, so it does not need rw_mutex and enters. As long as readers keep arriving one after another, read_count never reaches 0 and W1 keeps waiting — writer starvation.

Question 9: What are the roles of mutex and rw_mutex in the semaphore solution?

Answer: mutex only protects the variable read_count from concurrent updates by readers. rw_mutex controls access to the data: a writer takes it fully, while readers take it as a group (the first reader acquires it, the last releases it).

Question 10: Which variants suffer from starvation, and how can it be fixed?

Answer: Variant 1 starves writers and variant 2 starves readers. Variant 3 uses a service queue (a service semaphore, or a FIFO queue in a monitor) to preserve arrival order and has no starvation.

Question 11: (Multiple choice) Which method works for more than two processes and has no busy waiting? a) Dekker b) Peterson c) blocking semaphore d) none

Answer: Option c; Dekker and Peterson support only two processes and use busy waiting.

10. Conclusion

Dekker's and Peterson's algorithms show that mutual exclusion can be built with shared memory alone and no special instruction, but they are limited to two processes, use busy waiting and depend on correct memory ordering. Semaphores are scalable and efficient but low-level and error-prone. Monitors add structure and automatic mutual exclusion, making concurrent programming safer, and are very common in modern languages. The readers–writers problem shows that choosing the policy (reader priority, writer priority or fairness) matters as much as choosing the primitive.

- To learn correctness concepts: Dekker and Peterson.
- To control resources and execution order among many processes: semaphores.
- For structured, low-error code: monitors (or lock + condition variable).
- For shared data that is mostly read: the readers–writers pattern or pthread_rwlock_t.

In brief
Pick the primitive by the number of processes and the level of safety needed, and then pick the policy that matches the workload.

11. References

- Dijkstra, E. W. (1965). *Cooperating Sequential Processes* (EWD123). Technological University Eindhoven.
- Peterson, G. L. (1981). Myths About the Mutual Exclusion Problem. *Information Processing Letters*, 12(3), 115–116.
- Hoare, C. A. R. (1974). Monitors: An Operating System Structuring Concept. *Communications of the ACM*, 17(10), 549–557.
- Brinch Hansen, P. (1973). *Operating System Principles*. Prentice Hall.
- Courtois, P. J., Heymans, F., & Parnas, D. L. (1971). Concurrent Control with “Readers” and “Writers”. *Communications of the ACM*, 14(10), 667–668.
- Silberschatz, A., Galvin, P. B., & Gagne, G. *Operating System Concepts*. Wiley.
- Tanenbaum, A. S., & Bos, H. *Modern Operating Systems*. Pearson.